

A Federated Learning Approach Towards Cross-Architecture Ransomware Detection Using Low-Level Hardware Information

Chutitep Woralert

*Electrical and Computer Engineering
Clarkson University
Potsdam, New York, USA
woraltec@clarkson.edu*

Chen Liu

*Electrical and Computer Engineering
Clarkson University
Potsdam, New York, USA
cliu@clarkson.edu*

Abstract—The rapid proliferation of Malware-as-a-Service (MaaS) continues to generate diverse and evolving malware variants. This makes repeated and centralized retraining of malware detection models impractical, which also raises significant privacy concerns over the transmission of raw data needed for retraining. On the other hand, low-level hardware information has emerged as a powerful tool for dynamic malware analysis and detection, but how to offer behavioral insights across different processor microarchitectures remains a challenge. To address these challenges, we propose a federated learning framework for cross-architecture malware detection using low-level hardware information. By leveraging the inherent behaviors captured in architectural and microarchitectural event patterns, our approach enables malware detection models to generalize across heterogeneous environments while continually improving through distributed learning. Our experimental results show that the federated transfer learning model achieves an average detection accuracy of 96.88%, closely matching the performance of the traditional centralized fine-tuning model. These findings demonstrate that federated learning combined with low-level hardware information can provide an effective, scalable, and privacy-preserving foundation for cross-architecture malware detection in real-world distributed environments.

Index Terms—Hardware Performance Counter, Heterogeneous Architecture, Federated Learning, Malware Detection

I. INTRODUCTION

The appearance and rapid evolution of Malware-as-a-Service (MaaS) have made the production of malware and its variants from a daunting task to a more trivial one. This growing variability in malware significantly reduces the effectiveness of traditional static signature-based defenses, at the same time underscores the need for dynamic, behaviorally driven detection approaches that can automatically adapt to emerging threats. Among these threats, ransomware remains one of the most disruptive, encrypting user data and demanding payment for restoration, often causing severe financial and operational damage.

Low-level hardware information as performance monitoring events has recently emerged as a powerful tool for malware detection. Capturing and analyzing the hardware event activity during program execution can often expose behavioral patterns and reveal malicious intent as early indicators. Prior research

[1]–[4] has demonstrated that these hardware events, when combined with machine learning models such as long short-term memory (LSTM) networks or convolutional neural networks (CNNs), can detect diverse ransomware families with high accuracy. Similarly, cache-related hardware events have been shown to be highly effective in identifying side-channel attacks such as Meltdown and Spectre [5]–[7], which are notoriously difficult to detect using conventional methods.

Despite these promising results, two major challenges remain:

- i) Achieving scalability and generalization across heterogeneous processor architectures;
- ii) Updating detection models with new behavioral data without compromising user privacy.

Hardware event distributions can differ across processor architectures, making it difficult for models trained on one architecture to generalize to another. At the same time, transferring raw behavioral data for centralized retraining raises serious privacy concerns, especially with data from sensitive areas.

To address these challenges, in this paper, we explore federated learning as the foundation for a scalable and privacy-preserving malware detection framework that leverages shared behavioral patterns inherent in architectural and microarchitectural events. Federated learning enables a distributed model training across heterogeneous processors and, at the same time, ensures that raw data never leave the client machine. This design allows the global model to continuously improve and generalize without exposing sensitive user data.

Our experimental results show that the proposed federated transfer learning framework achieves an average detection accuracy of 96.88%, closely matching the performance of a centralized fine-tuning model while providing better scalability and privacy. Moreover, the model demonstrates strong generalizability, successfully detecting previously unseen ransomware variants across different processor architectures. The primary contributions of this work are summarized as follows:

- Demonstrate the scalability and adaptability of federated

transfer learning for cross-architecture malware detection, enabling rapid model adaptation across heterogeneous processor architectures while maintaining high accuracy.

- Demonstrate the effectiveness of the federated model in detecting previously unseen ransomware variants while achieving performance comparable to traditional centralized deep learning methods.

Together, these contributions establish federated learning combined with low-level hardware information as an effective, scalable, and privacy-preserving foundation for adaptive cross-architecture malware detection.

II. BACKGROUND

This section provides an overview of hardware performance counters and their relevance to ransomware and side-channel detection, followed by a summary of prior work in this domain.

A. Performance Monitoring Unit

Performance Monitoring Units (PMUs) are specialized hardware components found in modern processors that enable fine-grained monitoring of software and hardware execution behavior. By capturing a wide range of architectural and microarchitectural events through performance counters, PMUs offer a deep visibility into a system's internal activity. Because they introduce minimal overhead due to the utilization of dedicated hardware components, PMUs support continuous real-time profiling, making them suitable for applications such as performance optimization, workload characterization, anomaly detection, and security monitoring. Their ability to record low-level behavioral signals has made them particularly valuable for identifying malicious activity that may not be evident through higher-level system indicators.

B. Ransomware Attack and Detection

Ransomware is a type of malware that encrypts user files to deny access, then demands a ransom, typically in cryptocurrency, in exchange for the decryption key. Failure to comply often results in permanent data loss, which could have a significant financial and operational impact on victims. In recent years, ransomware has become increasingly prevalent and sophisticated [8], incorporating features such as full system lockdown, data exfiltration for double extortion, and evasion strategies to bypass detection mechanisms.

Given these evolving threat paradigm, researchers have explored the use of hardware performance counters as a behavioral signal for ransomware detection. Hardware event patterns often reflect the intensive file I/O and cryptographic operations characteristic of ransomware, which enables the possibility of early-stage detection.

Several studies have leveraged PMU data for this purpose. Alam et al. [1] introduced a one-class Long Short-Term Memory (LSTM) autoencoder that models benign hardware event patterns to detect deviations indicative of ransomware execution. Woralert et al. [2], [9] conducted detailed correlation analyses between hardware events and ransomware

activity to identify the most discriminative features for LSTM-based detection in both native and virtualized environments. Their work also evaluated detection performance against previously unseen variants and proposed mitigation techniques for ransomware employing evasion tactics. Ganfure et al. [3] transformed time-series performance counter data into image representations, which allowed models based on the convolutional neural network (CNN) to be used to learn spatial patterns and anomalies associated with ransomware behavior and achieve high detection accuracy.

C. Cache-Side Channel Attack

Cache side-channel attacks exploit the behavior of processor cache memory, which is designed to improve execution performance by storing frequently accessed data. By observing variations in cache access patterns and timing behavior, attackers can infer and leak sensitive information without directly compromising software vulnerabilities. Unlike conventional attacks that target program logic or protocols, side-channel attacks leverage indirect leakage sources such as execution timing, power consumption, electromagnetic emissions, or cache activity.

Common cache-based side-channel attacks include Spectre [5] and Rowhammer [10]. Spectre abuses speculative execution and branch prediction mechanisms to access and leak sensitive data from protected memory regions, while Rowhammer targets DRAM by repeatedly accessing specific memory locations to induce bit flips in adjacent rows. Many of these attacks rely on cache flushing and memory access patterns to perform the attack, making cache behavior a critical indicator of malicious activity.

Several prior studies have demonstrated the effectiveness of using performance counters to detect cache side-channel attacks by analyzing abnormal cache-related hardware event patterns generated during cache flushing and memory contention behaviors [6], [11]. By leveraging machine learning models to comprehend the anomalous patterns, these approaches can achieve high detection accuracy, highlighting the effectiveness of cache-level hardware events as reliable indicators for side-channel attack detection.

III. DETECTION MODEL FRAMEWORK

This section describes the proposed malware detection framework, which consists of two main components: the data collection module and the classifier module, as illustrated in Figure 1. We then present the model update strategies explored in this work, including traditional centralized transfer learning and the federated learning-based approach.

A. Data Collection Module

Performance counter data can be collected using a variety of profiling tools such as Perf Linux [12], K-LEB [13], Intel VTune [14], and Xcode Instruments [15]. In this work, we adopt K-LEB due to its lightweight kernel-level design with only 0.64% performance loss compared to 7.08% of Perf Linux [13]. This enables fast, precise, and low-overhead

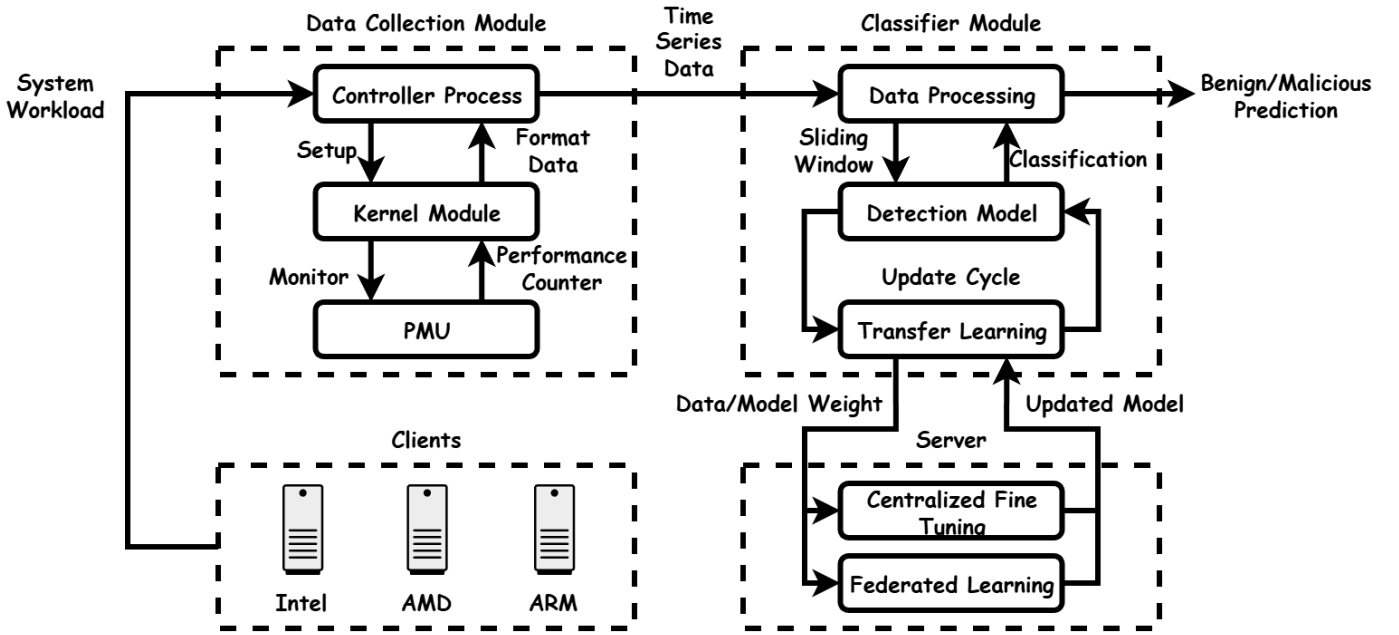


Fig. 1: Proposed Malware Detection Framework.

acquisition of hardware event data, which is essential for online application profiling and detection.

In this work, K-LEB is configured to periodically monitor system-wide user-level hardware events. K-LEB can operate directly on the user machine to observe architectural and microarchitectural activity without interfering with other running workloads. It additionally supports process-level monitoring, allowing the profiler to isolate measurements to a target application and its child processes, and thereby reducing noise from unrelated system activities.

To obtain time-series behavioral traces for classification, K-LEB samples hardware events every 10 ms, the recommended interval for fine-grain monitoring without incurring significant overhead. The data collected are then grouped on the basis of a 50-time-step sliding window. Each batch of time series data serves as an input instance to the classifier for real-time malware detection.

B. Classification Module

We employ two backbone machine learning models that have been shown to be very effective for processing time series data, Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM), to evaluate the effectiveness of hardware performance counter-based detection. The following subsections summarize the model structures used in this work.

1) *Convolutional Neural Network (CNN)*: CNNs are widely used to learn spatial patterns and have demonstrated a strong performance in computer vision and image processing tasks [16], [17]. They operate by applying learnable convolutional kernels to capture localized relationships within the input data. Compared to multilayer perceptrons (MLPs), CNNs restrict the flow of information to local receptive fields, reducing the

number of training parameters while improving their ability to extract distinctive features.

A convolutional layer works by convolving the input tensor with a learnable kernel to create the output. Given an input tensor $x \in \mathbb{R}^{c \times h \times w}$ with c denoting the number of channels and h and w denoting the height and width of the spatial dimensions, a convolutional kernel $w \in \mathbb{R}^{c \times k \times k}$ with kernel size k , and bias $b \in \mathbb{R}^{c \times h \times w}$, the output $y \in \mathbb{R}^{c \times h \times w}$ is defined as

$$y_{i,j} = b_{i,j} + \sum_{h=1}^c \sum_{a=-k}^k \sum_{b=-k}^k x_{c,i-a,j-b} w_{c,a,b}. \quad (1)$$

Pooling or strided convolutions are used to reduce spatial dimensions, and the final feature maps are flattened and fed into fully connected layers with a softmax classifier. Table I summarizes the CNN configuration used in our experiments, which employs a kernel size of 3, a channel size of 100, and ReLU activation, trained using categorical cross-entropy. The industry standard Adam optimizer is used for the optimization of the network [18].

2) *LSTM*: LSTMs [19] are a specialized class of recurrent neural networks (RNNs) designed to capture temporal dependencies in sequence data, while avoiding the vanishing gradient problem common to traditional RNNs [19]. To remedy this, the LSTM architecture introduces two states and three gates, see Equations (2) to (6) where σ denotes the activation function of the sigmoid, $W_{\cdot}$ denotes the weight tensors, $b_{\cdot}$

Layers	Hyperparameters	Shape
Input		(b, M, f)
Conv1D	$c = 25, k = 3$	$(b, M, 25)$
ReLU		$(b, M, 25)$
Flatten		$(b, 25M)$
Linear		$(b, 100)$
ReLU		$(b, 100)$
Linear		(b, N)
Softmax		(b, N)

TABLE I: Malware detection pipeline with CNN backbone. Let b denote the batch size, M denote the number of time-steps, and f denote the number of input features.

Layers	Hyperparameters	Shape
Input		(b, M, f)
LSTM	$c = 128$	$(b, M, 128)$
Dropout	$p = 0.5$	$(b, 128)$
Linear		$(b, 128)$
ReLU		$(b, 128)$
Linear		(b, N)
Softmax		(b, N)

TABLE II: Malware detection pipeline with LSTM backbone. Let b denote the batch size, M denote the number of time-steps, and f denote the number of input features.

denotes the bias tensors, and $\odot$ is the Hadamard product.

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i) \quad (2)$$

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f) \quad (3)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o) \quad (4)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c) \quad (5)$$

$$h_t = o_t \odot \tanh(c_t) \quad (6)$$

The initial states are given as $c_0 = 0$ and $h_0 = 0$. The hidden state, h_t , is used to store encoded information about the previous time step; conversely, the cell state, c_t , acts as a form of global memory of the LSTM network on all time steps. Every gate operates on the current data, x_t , and the hidden state of the previous time step, h_{t-1} . The input gate, i_t , is used to determine how much of the current data will be remembered in the global memory, c_t . Likewise, the forget gate, f_t , is used to determine how much global memory should remember past events. Finally, the output gate, o_t , determines how strongly the output signal, $\tanh(c_t)$, should be passed to the next hidden state and the output, h_t .

The LSTM model is trained to predict the class of the given time series data. Table II shows the LSTM backbone for the two-class approach consisting of an LSTM layer of 128 units followed by a dropout layer and two dense fully connected layers. After the output of the LSTM model, a two-layer dense feedforward neural network is attached to the top of the first layer using the ReLU activation function and a softmax activation function in the last layer to scale the output to class probabilities. Like with the CNN model, the network is then trained to minimize cross-entropy with Adam optimizer.

C. Transfer Learning Methods

To adapt the detection framework to a new deployment environment, it is necessary to update the classifier using hardware event data collected from the target machine. Rather than retraining the model from scratch, we apply a transfer learning technique to existing pre-trained CNN and LSTM models, which serve as a baseline and are fine-tuned using benign data from the new system. This approach reduces the need to collect additional malicious samples, which are generally more difficult and risky to obtain.

However, centralized fine-tuning requires that the new data be transmitted to a central server, which can raise privacy concerns and increase the likelihood of data exposure during transfer. To address this issue, we adopt a federated learning-based transfer learning approach. As illustrated in Figure 2, instead of sharing the data between the client and the server, the central server distributes the baseline model to participating clients. Each client fine-tunes the model locally using its own data, ensuring that sensitive raw data never leave the device. The clients then return only updated model parameters to the server, where they are aggregated. In this work, we used Federated Averaging (FedAvg) [20], implemented via the Flower framework for model aggregation. Once the aggregation process is finished and the central baseline model is updated based on the new parameters, the central server redistributes the updated model to the clients for the next iteration. With every iteration, the models gather a varied amount of information and improve further without creating data privacy breaches, as the data were never shared during the model update.

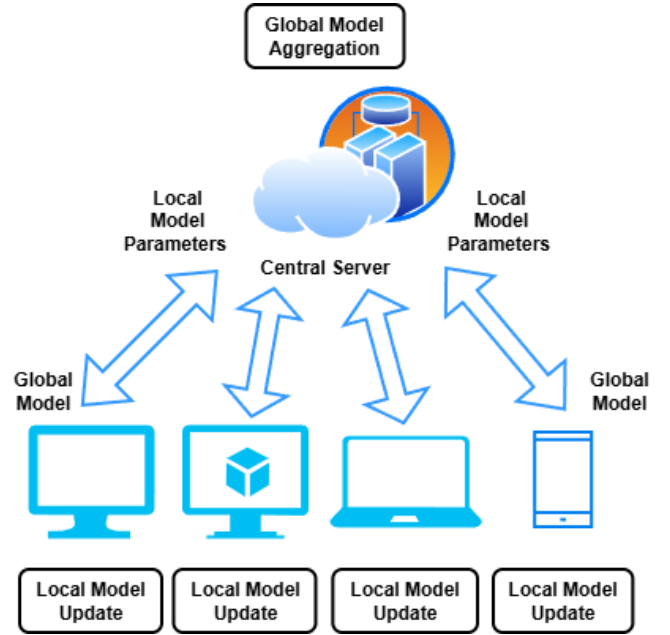


Fig. 2: Federated Learning Framework.

This iterative process allows the global model to integrate diverse behavioral patterns across clients while preserving data

privacy. Only model updates are exchanged, not datasets. The federated learning approach significantly reduces bandwidth consumption and supports large-scale deployments across heterogeneous devices.

Both traditional centralized transfer learning and federated learning enable efficient model adaptation and mitigate concept drift, but the federated learning approach offers stronger privacy and scalability for distributed malware detection systems.

IV. EXPERIMENT ANALYSIS

In this section, we evaluate the performance of the proposed detection framework in terms of its ability to accurately identify ransomware and side-channel attacks. Furthermore, we analyze the effectiveness of transfer learning strategies in adapting the classifier to heterogeneous deployment environments and in detecting previously unseen ransomware variants. Through these evaluations, we assess both detection performance and model generalization across diverse system behaviors.

A. Experiment Setup

To evaluate the effectiveness and cross-architecture generalization of the proposed detection framework, experiments were conducted on a diverse set of client machines spanning multiple processor architectures and deployment environments. The experiments were performed on:

- i) Intel Core i7-920 @ 2.67GHz processor with 6GB DDR3 memory running Ubuntu 16.04 with Linux kernel version 4.13.0-15,
- ii) AMD Ryzen-7 5800X 3.8GHz 8-Core Processor with 32GB memory running Ubuntu 20.04 with the Linux kernel version 5.15.0-52,
- iii) Raspberry Pi 4 Model B Broadcom BCM2711 @ 1.5GHz processor with 4GB memory running Debian 10 with Linux kernel version 5.4.72-v8+.

In addition to edge and desktop platforms, cloud-based environments were evaluated using Amazon Web Services (AWS), including

- iv) Intel Xeon Platinum 8259CL @ 2.50GHz processor running Ubuntu 16.04 with Linux kernel version 4.4.0-1112-aws,
- v) AWS Graviton Processors feature 64-bit Arm Neoverse with 32GB memory running Ubuntu 20.04.2 LTS with Linux kernel version 5.4.0-1041-aws.

These client machines are configured as isolated sandboxes for malware execution. The detection framework was deployed locally on each client system to monitor and collect performance counter data. Distinct benign workloads were assigned to each environment to simulate different system behaviors. The benign behavior included common Linux commands, web browsing, web hosting, video streaming, machine learning related tasks, and network monitoring tools.

In this experiment, we used ransomware samples from various ransomware families, including *Alphv*, *Blackcat*, *Conti*,

Hellokitty, *Holycrypt*, *Monti*, *RansomEXX*, *REvil*, and *Sodinokibi*, obtained from the malware database [21]. In addition, several open-source ransomware implementations, such as *C Ransomware* [22], *Cryptsky* [23], and *ransomware PoC* [24], were also included. For side-channel evaluation, an exemplary Specter attack was downloaded from publicly available proof-of-concept implementations [25], then executed inside the client machines to collect their behavioral traces on the system.

Ransomware attacks primarily involve intensive encryption and file system activity, while Spectre exploits speculative execution and cache behavior. Based on these characteristics, five architectural and microarchitectural hardware events were selected as input features: branch instructions, branch misses, L2 cache references, L2 cache misses, and instruction retires. These events are commonly available across processor architectures and can capture consistent behavioral patterns relevant to both ransomware and cache-based side-channel attacks. The performance counter data collected were used to train and evaluate the CNN- and LSTM-based classifiers described in Section III-B.

B. Transfer Learning Applications and Results

1) *Transfer Learning for Generalization:* In this experiment, we evaluated the generalizability of models trained on local client data when deployed across different processor architectures and deployment environments. Figure 3 illustrates the model notation used for this experiment. The baseline models were first constructed: the Intel model trained solely on local Intel machine data, the AMD model trained solely on local AMD machine data, and the ARM (Pi) model trained solely on local ARM Raspberry Pi machine data. Similarly, the AWS machines also have their own baseline model trained exclusively on the data from their Intel (AWS) machine and ARM (AWS) machine. To improve adaptability, we then employ two transfer learning methods: centralized transfer learning (CL) and federated learning (FL) to fine-tune the baseline model. The CL model is built using the centralized training method with the Intel model as a baseline model; then we fine-tuned the baseline model to include additional data from other clients. The FL model was trained using the federated learning strategy, where a shared global model was distributed to all clients for local training, and updated local weights were aggregated on a central server to produce the refined global model.

Figure 4 shows the results of ransomware detection performance using CNN and LSTM models for different environments. As expected, CNN models trained locally achieve strong performance in their native environments, with detection accuracy over 99.72% on average across all baseline models. However, when deployed in non-native environments, the performance of the model drops significantly, e.g., the detection accuracy of the ARM (Pi) model falls to 59.54% and 52.69% when tested in the Intel and AMD environments. Similar results can be observed with Intel and AMD models tested in ARM (Pi) environment, with detection accuracy dropping to 73.64% and 56.89%, respectively. The model

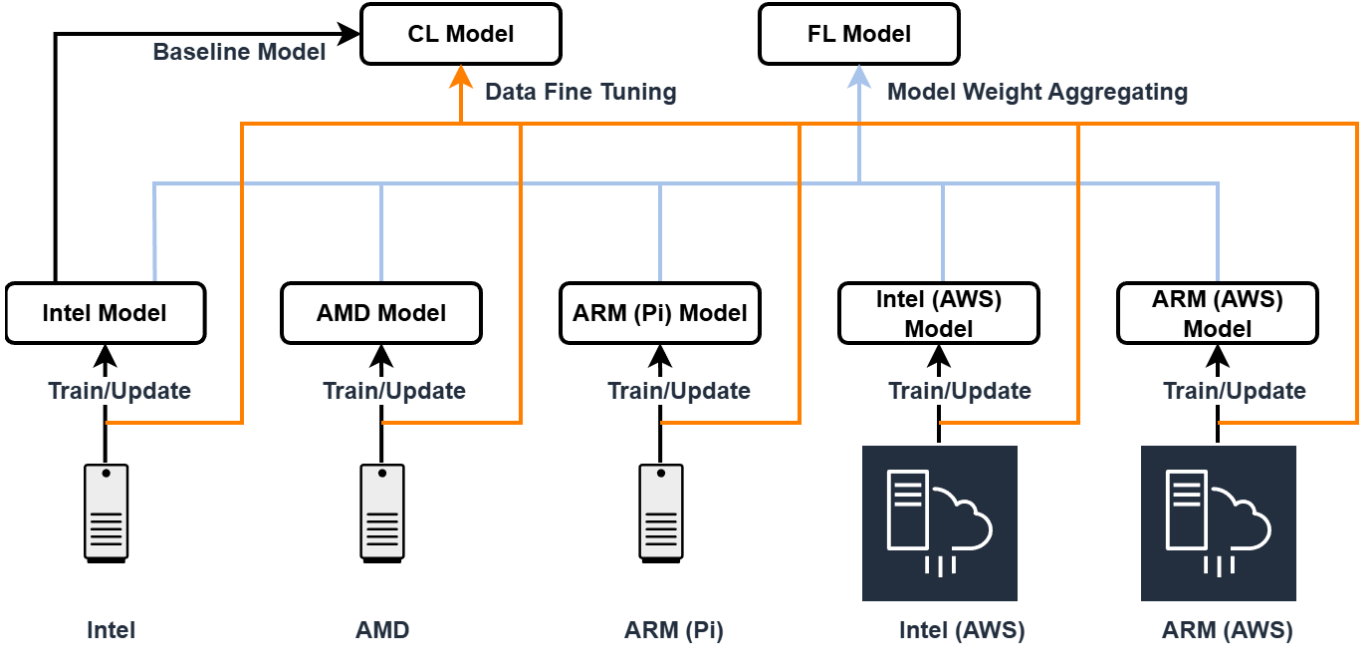


Fig. 3: Models Notation for Experiments.

seems to have difficulty differentiating between different benign system behaviors from different environments, as shown in the significant drop in overall precision values. In contrast, ransomware behavior can be observed across different environments and architectures with a relatively high recall rate being maintained. A similar observation can be seen with the LSTM model.

Applying centralized fine-tuning substantially improves generalization across architectures. The centralized fine-tuned model is able to detect the ransomware attack with very high accuracy and consistently across different environments, with a detection accuracy over 99.31% on average for the CNN model, while LSTM also achieves a similar detection accuracy of 99.28% on average. This shows the capability of the centralized training method to fine-tune the model to the new environment. What we found impressive is that the federated learning model is able to achieve strong results, with detection accuracy of 97.94% on average for the CNN model. This highlights the adaptability of the federated learning model to new environments. However, the LSTM model sees a minor drop in performance for the federated learning approach with a detection accuracy of 94.97%.

Figure 5 shows the results of Spectre side-channel attack detection performance for different environments. Similar generalization trends are observed; however, Spectre exhibits more distinctive hardware-level signatures between benign and malicious cases compared to ransomware. As a result, both baseline and transfer learning models maintain higher precision overall, reflecting the clearer separation between benign and malicious behaviors in cache-based side-channel attacks.

Finally, we evaluate a unified multi-class model that jointly detects ransomware and Spectre attacks. As shown in Figure 6, the combined model achieves a performance comparable to individual binary classifiers, demonstrating that multiple types of attacks can be detected within a single model without a significant loss in accuracy.

Although the federated learning approach occasionally yields slightly lower accuracy than centralized fine-tuning, it provides a critical advantage in preserving data privacy by eliminating the need to share raw performance counter data. This balance between strong detection performance and enhanced privacy protection positions the federated learning approach as a practical and scalable solution for cross-architecture malware detection in real-world distributed environments.

2) *Detection of unseen attack variants*: To evaluate the robustness of the proposed framework against previously unseen ransomware variants, we analyze recall rates across a diverse set of ransomware families, as summarized in Table III. In this experiment, each baseline model is evaluated on ransomware families that were not present in its local training, thereby assessing its ability to generalize beyond known attack instances. CNN- and LSTM-based classifiers are considered across heterogeneous processor architectures, including Intel, AMD, ARM (Raspberry Pi), and cloud-based platforms (AWS).

The results show that baseline models trained on native environments exhibit highly inconsistent performance when detecting unseen ransomware variants across architectures. For CNN-based models, the Intel and AMD baselines achieve relatively high recall for most families, with recall values

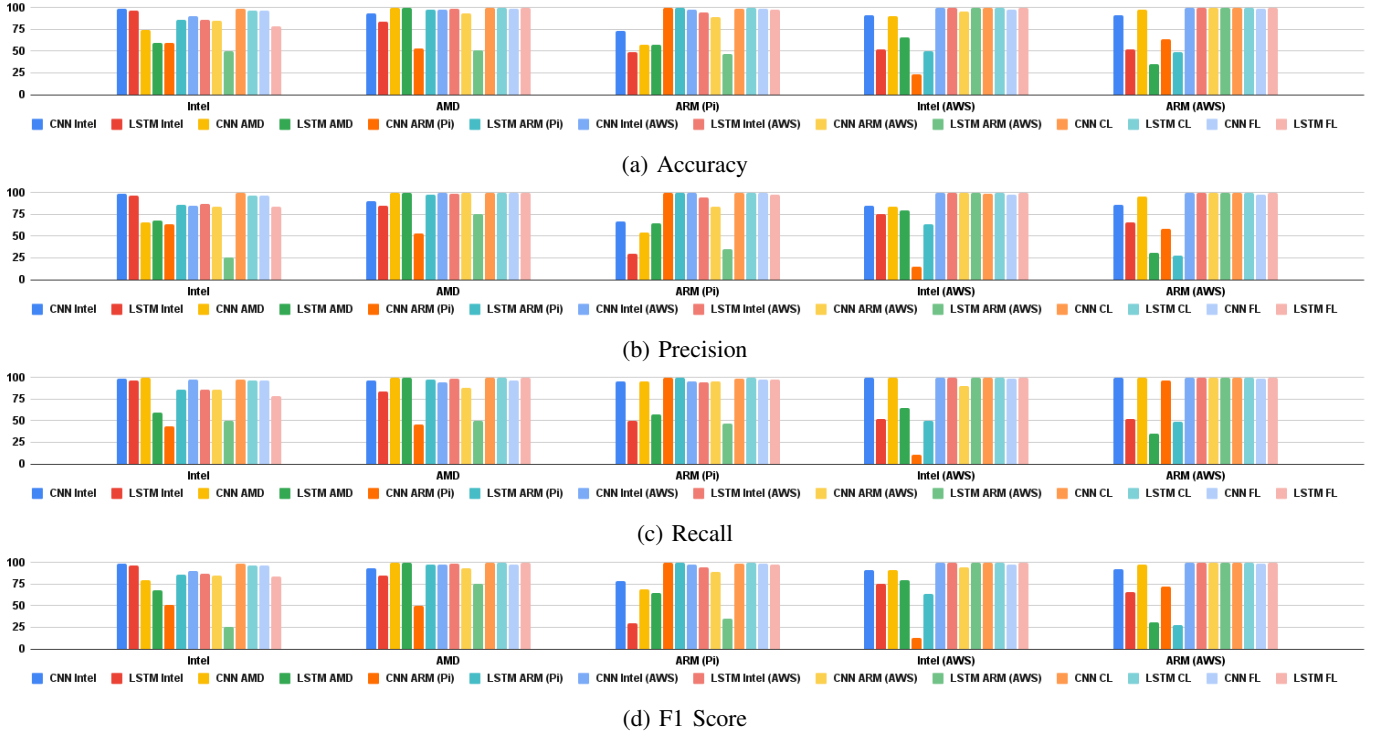


Fig. 4: Models Detection Results for Ransomware Attack on Heterogeneous Processor Architectures.

frequently exceeding 94%. In contrast, the baseline model of the ARM (Pi) exhibits substantial recall degradation for multiple ransomware families, such as *Alphv* (43.54%), *Blackcat* (34.16%), *Conti* (14.51%) and *REvil* (30.91%). This is potentially due to the limited ransom attack data on Raspberry Pi with only two families *Cryptsky* and *PoC*, which causes the ARM (Pi) model not to be able to generalize. Similar performance variability is observed in the ARM (AWS) baselines, which also have limited ransomware attack data, indicating that models trained on a single platform with limited ransomware data struggle to generalize when confronted with unseen attack characteristics.

Despite this variability, certain ransomware families, such as *Hellokitty*, *RansomExx*, *Sodinokibi*, and *ransomware PoC*, maintain high recall across most environments even in unseen cases. This observation suggests that these families exhibit strong, shared behavioral signatures with encryption activity patterns observable through performance counters with different families.

Applying centralized transfer learning (CL) significantly improves recall consistency across all ransomware families and architectures. The centrally fine-tuned CNN model achieves near-perfect recall, exceeding 97% for all ransomware variants, including those that previously caused severe misclassification in the baseline models. Similarly, the CL-based LSTM model demonstrates uniformly high recall, with values approaching or reaching 100% across all families. These results confirm that the aggregation of diverse behavioral data enables the model to better capture the generalized characteristics of

ransomware.

The federated learning (FL) approach also delivers strong performance, achieving recall rates that closely match those of centralized learning. For CNN models, the FL approach maintains recall above 82% for all variants and exceeds 93% for most families, including challenging cases such as *Cryptsky* and *Conti*. Similarly, the FL-based LSTM model consistently achieves high recall across all ransomware families, often reaching 100%. Although federated learning exhibits a small reduction in recall compared to centralized fine-tuning in certain cases, overall performance remains robust and significantly superior to baseline models.

Overall, these results demonstrate that while locally trained models struggle to detect unseen ransomware variants across heterogeneous architectures, both centralized and federated transfer learning substantially enhance generalization. Importantly, the federated learning approach achieves this improvement without requiring raw data sharing, thereby preserving client privacy. This ability to generalize variants of ransomware behavior patterns across diverse architectures highlights the practicality and resilience of the proposed federated learning framework for real-world malware detection scenarios.

V. DISCUSSION

In this section, we discuss possible future directions for the proposed federated framework.

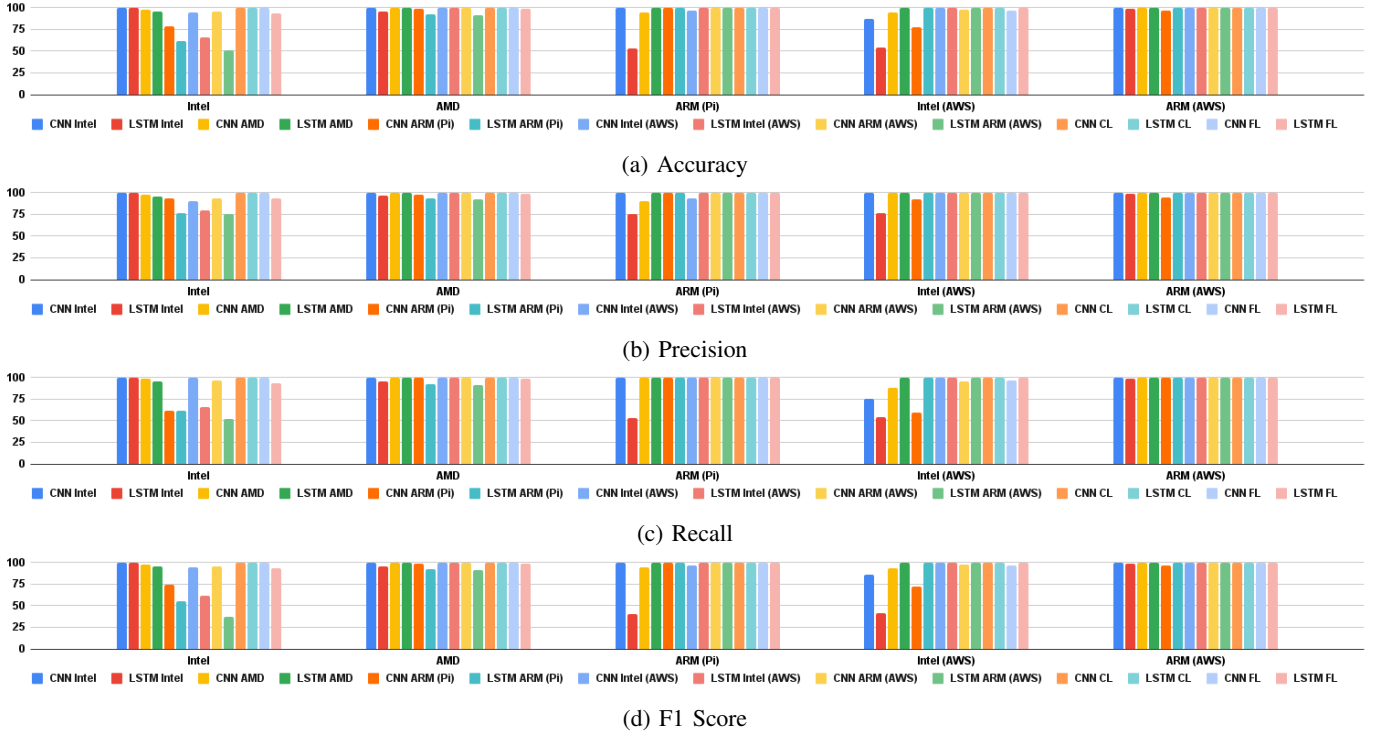


Fig. 5: Models Detection Results for Spectre Attack on Heterogeneous Processor Architectures.

TABLE III: Transfer Learning Recall Rates for Ransomware Variants.

Ransom Families	Alphv ²	Blackcat ^{1,2}	C ⁵	Conti ^{1,2}	Cryptsky ^{3,5}	Hellokitty ²	Holycrypt ⁵	Monti ²	PoC ^{3,5}	RansomExx ^{1,2,4}	REvil ^{1,2,4}	Sodinokibi ^{1,2,4}
CNN Models												
Intel ¹	94.64	95.68	100.00	94.16	97.57	99.57	100.00	92.04	97.82	99.75	100.00	100.00
AMD ²	100.00	99.46	100.00	100.00	98.23	100.00	100.00	99.49	98.13	99.93	100.00	100.00
ARM (Pi) ³	43.54	34.16	65.96	14.51	99.98	65.16	100.00	3.66	99.97	30.91	43.27	41.44
Intel (AWS) ⁴	96.32	93.13	99.96	88.28	97.99	97.71	100.00	65.22	98.04	98.32	100.00	100.00
ARM (AWS) ⁵	86.14	77.49	100.00	79.01	97.88	82.30	100.00	49.34	97.93	81.40	99.62	99.07
CL ^{1,2,3,4,5}	100.00	98.77	100.00	99.75	99.15	100.00	100.00	97.33	99.23	99.78	100.00	100.00
FL ^{1,2,3,4,5}	96.52	93.98	100.00	89.53	98.76	99.64	100.00	82.42	98.70	96.61	100.00	100.00
LSTM Models												
Intel ¹	90.22	93.66	94.25	84.00	99.66	98.91	85.91	84.12	99.51	97.33	99.89	98.59
AMD ²	99.01	96.29	99.97	95.46	46.63	100.00	0.00	100.00	47.90	99.11	100.00	100.00
ARM (Pi) ³	92.79	85.63	97.18	83.59	99.98	99.46	59.76	99.25	99.94	94.2	99.91	99.82
Intel (AWS) ⁴	91.17	87.30	99.94	98.01	94.17	100.00	88.99	100.00	94.10	99.26	99.98	100.00
ARM (AWS) ⁵	100.00	100.00	100.00	100.00	95.32	100.00	100.00	100.00	95.58	100.00	100.00	100.00
CL ^{1,2,3,4,5}	99.33	96.34	99.90	98.61	100.00	99.79	100.00	99.90	100.00	98.11	99.90	99.93
FL ^{1,2,3,4,5}	95.49	93.74	100.00	100.00	96.61	100.00	100.00	100.00	97.12	100.00	100.00	100.00

A. Privacy and Security

Although federated learning significantly reduces privacy risks by ensuring that raw data remain on client devices, the possibility of information leakage through model gradients or update patterns still poses a concern. Techniques such as differential privacy and secure aggregation could be integrated to further minimize this risk of model inversion or reconstruction attacks. Furthermore, the current setup assumes trusted clients; however, real-world deployments must account for adversarial participants capable of data poisoning or manipulating model updates. For that reason, robustness against data poisoning and adversarial updates should be implemented and evaluated. Future work should explore defenses against adversarial attacks to ensure resilience against malicious participants.

B. Heterogeneous Feature Data

Although hardware performance counter data have demonstrated strong effectiveness in detecting ransomware and side-channel attacks, relying solely on low-level hardware events may not be sufficient against increasingly sophisticated and evasive malware. Advanced malware variants may deliberately alter their execution behavior to reduce abnormal hardware activity and evade performance counter based monitoring. For example, ransomware employing padding or workload obfuscation techniques can inject benign instructions or throttle encryption activity to mask the characteristic behavioral patterns typically exposed through hardware events.

Another limiting factor arises from the number of hardware performance counters that can be monitored concurrently.

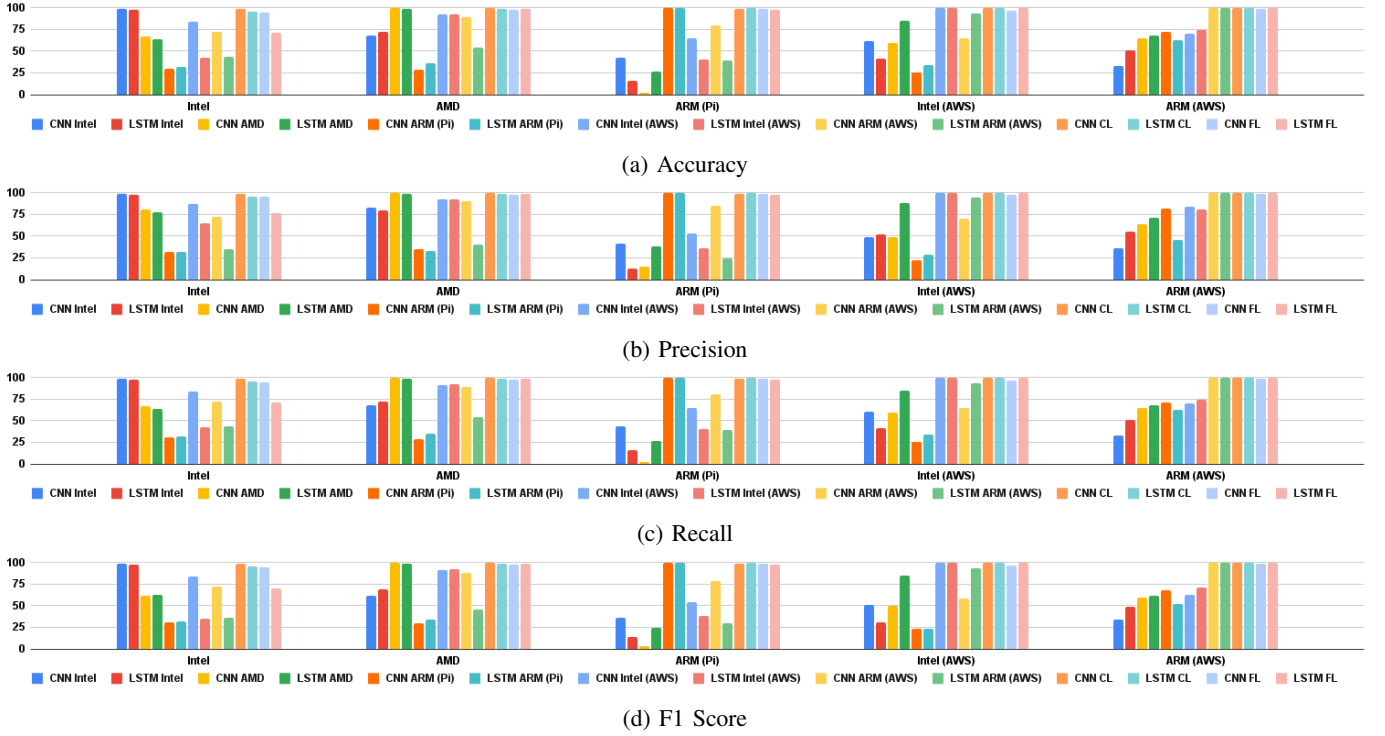


Fig. 6: Models Detection Results for Multi-Class Model on Heterogeneous Processor Architectures.

Modern processors provide only a limited number of hardware registers for hardware event collection, restricting the amount of architectural and microarchitectural information that can be observed at any given time. To mitigate this limitation, multiplexing techniques can be employed to periodically switch between different hardware events to enable the monitoring of a larger set of counters than the number of physically available registers. However, multiplexing may introduce sampling inaccuracies and temporal inconsistencies, specifically to detect short-lived or rapidly evolving attack behaviors.

To further improve detection coverage, future work should explore heterogeneous and multi-modal feature integration. Combining hardware performance counter data with complementary information sources, such as network traffic activity, memory access patterns, file system behavior, and system call traces, could provide a more comprehensive representation of malicious behavior. Such multi-modal approaches may improve resilience against evasive malware techniques by capturing behavioral indicators that are not directly observable through hardware events alone. In particular, integrating heterogeneous data features with the federated learning framework could enhance generalization across diverse attack scenarios while maintaining privacy-preserving distributed training.

C. Scalability and Deployment

The current implementation was evaluated on various clients with heterogeneous processor architectures to demonstrate the proof-of-concept for federated learning performance. Ex-

tending the setup to a larger number of clients will be an important next step to assess its scalability, communication overhead, and convergence behavior in realistic federated networks. Additionally, exploring asynchronous, hierarchical, or communication variants of federated learning could further reduce latency and bandwidth usage, enabling a more practical deployment in large-scale distributed environments.

Despite its advantages, federated learning introduces practical challenges in deployment and management. Coordinating distributed clients, handling non-independent and non-identically distributed (non-IID) data, and ensuring stable model convergence can be complex in large-scale settings. Data heterogeneity across clients may lead to slower convergence or degraded performance if not properly addressed. Future work will investigate adaptive aggregation strategies, client clustering, and personalized federated learning techniques to mitigate these issues.

VI. CONCLUSIONS

In this paper, we demonstrate the effectiveness of leveraging low-level hardware information as a foundation for a dynamic malware detection framework. By capturing shared architectural and microarchitectural behavioral patterns, our approach effectively addresses the challenge of adapting detection models across heterogeneous environments. We introduce a federated transfer learning framework that enables scalable and privacy-preserving model adaptation without requiring raw performance counter data to be shared. Extensive experimental evaluation across diverse processor architectures, including

desktop, edge, and cloud platforms, shows that the proposed federated approach achieves an average detection accuracy of over 96.88% on average, closely matching the performance of centralized fine-tuning. Moreover, our results demonstrate strong generalization in detecting multiple ransomware families and cache-based side-channel attacks, demonstrating the transferability of hardware-level behavioral signatures across architectures.

While centralized fine-tuning yields slightly higher accuracy in some cases, the federated learning approach offers a compelling trade-off by maintaining high detection performance while preserving user data privacy and reducing communication overhead. This balance makes federated learning particularly well-suited for real-world deployment in distributed and heterogeneous systems, where data sharing is constrained by privacy and security concerns.

Given these promising results, several opportunities for future work remain. Incorporating additional privacy enhancement mechanisms, such as differential privacy and secure aggregation, can further mitigate risks associated with model inversion and adversarial manipulation. Evaluating scalability, communication efficiency, and robustness across larger client populations and more diverse processor families will be essential to validate real-world applicability. Finally, integrating multi-modal features, such as network traffic and memory access patterns, may further enhance detection accuracy and interpretability, enabling a more comprehensive and resilient malware detection framework capable of defending against increasingly sophisticated cyber-threats.

REFERENCES

- [1] M. Alam, S. Sinha, S. Bhattacharya, S. Dutta, D. Mukhopadhyay, and A. Chattopadhyay, "Rapper: Ransomware prevention via performance counters," 2020.
- [2] C. Woralert, C. Liu, and Z. Blasingame, "Hard-lite: A lightweight hardware anomaly realtime detection framework targeting ransomware," *IEEE Transactions on Circuits and Systems*, pp. 1–12, 2023.
- [3] G. O. Ganfure, C.-F. Wu, Y.-H. Chang, and W.-K. Shih, "Deepware: Imaging performance counters with deep learning to detect ransomware," *IEEE Transactions on Computers*, vol. 72, no. 3, 2023.
- [4] P. M. Anand, P. V. S. Charan, and S. K. Shukla, "Hiper - early detection of a ransomware attack using hardware performance counters," *Digital Threats*, vol. 4, oct 2023.
- [5] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, "Spectre attacks: Exploiting speculative execution," in *40th IEEE Symposium on Security and Privacy (S&P'19)*, 2019.
- [6] M. Mushtaq, J. Bricq, M. K. Bhatti, A. Akram, V. Lapotre, G. Gogniat, and P. Benoit, "Whisper: A tool for run-time detection of side-channel attacks," *IEEE Access*, vol. 8, pp. 83871–83900, 2020.
- [7] W. Wang, G. Chen, Y. Cheng, Y. Zhang, and Z. Lin, "Specularizer : Detecting speculative execution attacks via performance tracing," in *Detection of Intrusions and Malware, and Vulnerability Assessment* (L. Bilge, L. Cavallaro, G. Pellegrino, and N. Neves, eds.), pp. 151–172, Springer International Publishing, 2021.
- [8] Fortinet, "Ransomware statistics and ransomware trends 2025."
- [9] C. Woralert, C. Liu, and Z. Blasingame, "Towards effective machine learning models for ransomware detection via low-level hardware information," in *Proceedings of the International Workshop on Hardware and Architectural Support for Security and Privacy 2024*, HASP '24, (New York, NY, USA), p. 10–18, Association for Computing Machinery, 2024.
- [10] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, "Flipping bits in memory without accessing them: An experimental study of dram disturbance errors," in *Computer Architecture (ISCA), 2014 ACM/IEEE 41st International Symposium on*, pp. 361–372, June 2014.
- [11] J. Depoix and P. Altmeyer, "Detecting spectre attacks by identifying cache side-channel attacks using machine learning," p. 11.
- [12] A. C. De Melo, "The new linux'perf'tools," 2010.
- [13] C. Woralert, J. Bruska, C. Liu, and L. Yan, "High frequency performance monitoring via architectural event measurement," in *2020 IEEE International Symposium on Workload Characterization (IISWC)*, pp. 114–122, 2020.
- [14] "Intel vtune profiler," 2022.
- [15] "Xcode instruments," 2019. [Online]. <https://help.apple.com/instruments/mac/current/>.
- [16] K. Fukushima, "Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position," *Biological Cybernetics*, vol. 36, pp. 193–202, 1980.
- [17] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems* (F. Pereira, C. Burges, L. Bottou, and K. Weinberger, eds.), vol. 25, Curran Associates, Inc., 2012.
- [18] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *CoRR*, vol. abs/1412.6980, 2015.
- [19] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, pp. 1735–1780, 1997.
- [20] Flower, "Flower a friendly federated ai framework."
- [21] "Malwarebazaar," 2024.
- [22] D. Affinita, "Ransomware written in c," 2020.
- [23] S. Curtis, "Cryptsky," 2017.
- [24] Jimmy-ly00, "Ransomware-poc," 2020.
- [25] E. A. Johnson, "Spectre proof-of-concept." <https://gist.github.com/ErikAugust/724d4a969fb2c6ae1bbd7b2a9e3d4bb6>.